TwinEval AI - Smart Evaluation of Handwritten Answer Scripts

1.Ashwin Kirbaa B 2.Akhileshkar T P 3.Alen K R 4.Aswin R Mrs S V Hemalatha

Department of Computer Science and Engineering, Sri Shakthi Institute of Engineering and Technology, Coimbatore, India

ABSTRACT

Background: TwinEval AI is a web-based prototype that evaluates handwritten answer scripts against an uploaded question paper and allocates marks question by question. Objective: To describe the inspected system and report preliminary behaviour of its scoring pipeline on deterministic component checks and stored pilot records. Methods: Source-code inspection covered the Node.js/Express and SQLite backend, the Python AI engine, handwriting extraction, answer-key generation, rubric scoring and the teacher review workflow. Component checks exercised grade mapping, objective-answer matching, question splitting, answer splitting and fallback rubric scoring; the stored records of one 20-question paper and one scanned script evaluated twice were also examined. Marks were not compared with independent examiner marks. Results: The same scanned script received 39.53/50 in one run and
35.45/50 in another, with all 12 objective items identical and the largest single-question difference at 2.85 marks. Grade mapping passed 10/10 cases, objective matching 9/10, question splitting 4/4 and answer splitting 3/4. Conclusion: These results describe system behaviour and run-to-run variation. They do not establish marking accuracy, agreement with teachers, OCR quality or fairness. Calibration against independent examiners, OCR benchmarking and usability evaluation remain necessary.

KEYWORDS

Handwritten answer evaluation, optical character recognition, local language models, rubric-based grading, educational technology, artificial intelligence.

1. INTRODUCTION

Examination marking in higher education remains largely manual: scripts are handwritten, marked under time pressure and judged against rubrics that different examiners apply with different strictness. Automatic short-answer and essay scoring has been studied for decades, and reviews report that results depend heavily on the task, the reference material and the scoring method [1,2,17,21]. Handwritten scripts add a further step, because text must first be recovered by optical character recognition (OCR) before any scoring is possible [10]. Language models can now read and grade free text, but their judgements can vary between runs and call for human oversight [5,12]. Student scripts are also sensitive records, which motivates running models on institutional hardware rather than on third-party servers.

TwinEval AI is a web prototype that accepts a question paper and a scanned handwritten answer script, generates an answer key, extracts the handwriting and allocates marks for each question. This report describes the system architecture and a limited set of deterministic checks and pilot records, without claiming agreement with human examiners.

2. LITERATURE SURVEY

Automatic short-answer grading compares a student response with a reference answer or rubric, using surface overlap, semantic similarity or learned models [1]. Reviews of automated essay scoring show that accuracy varies with dataset and metric [2]. Sentence embeddings allow meaning-level comparison of two texts [3], and compact distilled encoders such as MiniLM make this practical on a laptop CPU [4]. Agreement between graders is commonly reported with quadratic weighted kappa [9,20] and automated scoring guidance asks for validity evidence beyond agreement [8].

Transformer-based [22] instruction-tuned language models are increasingly used as judges; studies find useful agreement with humans but also position, verbosity and self-preference bias [5]. Reviews of language models in education add concerns about accuracy and privacy [12]. Open-weight models such as Gemma 3 [6], served through Ollama [11], allow local execution.

2. 1 Project context and gap

TwinEval AI combines OCR, language-model answer keys, rubric scoring and teacher review. The present report addresses a documentation gap by separating implemented components from unverified claims and by reporting deterministic component checks and stored pilot records. It does not claim that these establish marking accuracy or superiority to other systems.

3. PROPOSED METHODOLOGY

3.1 System architecture

The inspected application has three tiers (Table 1). A browser client of 15 static pages communicates over REST with a Node.js/Express server that stores data in a portable SQLite database through the built-in node:sqlite module [14]. Authentication uses JSON Web Tokens [15] and bcrypt password hashing, with mandatory e-mail verification through SMTP and student, teacher and admin roles; this description is not a security assessment. Evaluation is delegated to a Python/Flask AI engine on port 8000, which runs background jobs that the client polls. The engine calls a local Ollama service [11]; the project defaults to gemma3:4b [6] through the AI_ENGINE_MODEL setting. The model digest, quantisation and hardware used for the stored pilot runs were not recorded.

The design intends that student data stay on local hardware, but this holds only for the default path. If a Gemini API key is configured, answer-key generation can use a remote model and OCR can fall back to it, so exam content may then leave the device. Complete disconnected operation was not demonstrated.

3.2 Question paper processing and answer keys

A teacher types or uploads a question paper as PDF, DOCX, TXT or image. PyMuPDF [16] reads the embedded text of a PDF page when it appears genuine; scanned or garbled pages are rasterised and passed to OCR. The question agent first splits the text with regular expressions that accept markers such as “1.”, “Q1)” and “Question 1:”, and calls the language model only when too few questions are found. Marks come from an inline annotation such as [10 Marks] or from a section header such as 12×0.5, and questions can belong to a choice group in which only a required number is counted.

For every question an answer-key agent generates a structured key: model answer, keywords, concepts, expected points, advantages, disadvantages, applications, formulae, steps, diagrams, a Bloom’s taxonomy level [7,19] and a rubric whose criteria sum to the question’s maximum marks. Keys are stored when the teacher finalises the paper. Their correctness was not independently verified.

Table 1. Component inventory in the inspected project snapshot.

Component group	Count	Source/status
AI agent modules	14	ai-engine/agents (Python)
Express route definitions	51	src/routes (Node.js)
AI engine endpoints	13	ai_server.py (Flask)
Database tables	10	SQLite schema, created at start-up
Web pages	15	public/ static pages
Bloom levels / keywords	6 / 48	bloom_levels.json
Calibration exemplars	0	File present but empty

The tables include Users, Student, QuestionPaper, Question, AnswerKey, Submission and QuestionResult. The empty calibration file means that no teacher-scored examples anchored the pilot runs. Bloom levels are inferred by counting keyword matches, not by a trained classifier.

3.3 Handwriting extraction and evaluation

Student scripts are uploaded as PDF or images. Text is extracted with a fallback chain: a Gemma vision model through Ollama, then Gemini OCR if a key exists, then PaddleOCR [18]. A utility for grayscale conversion, adaptive thresholding and resizing to at most 1,920 pixels is available. A student-answer agent then splits the combined text into per-question answers using markers such as “Ans 1:”, with a model fallback.

Each answer is then evaluated in order. (i) Items worth 1 mark or less use an objective path: full marks if the option letter, true/false value or key phrase matches, otherwise zero. (ii) Longer answers pass a topic-relevance gate, and a relevance score below 0.15 gives zero marks. (iii) Concepts, keywords, expected points and other key elements are matched using stemmed terms. (iv) A holistic model judgement at temperature 0.2 scores each rubric criterion, optionally anchored by up to three teacher-scored exemplars. (v) If that call fails, a mechanical rubric score applies the square root of the coverage ratio to each criterion, with completeness weighted 0.45 for concepts, 0.45 for points and 0.10 for length. Grades follow fixed bands from A+ at 90% to F below 40%. Confidence is 60 + 40 × relevance, capped at 100.

3.4 Teacher review and safeguards

A result is flagged for teacher review when the score lies within five percentage points of the pass line, when relevance is between 0.3 and 0.55, or when confidence is below 55. Teachers can override the overall score or an individual question, and any question can be flagged to a persistent queue whose resolved entries are intended to form a calibration set. An offline harness computes quadratic weighted kappa, mean absolute error, Pearson correlation and bias by score band, but it was not run here because no resolved flags or exemplars were available. A ReportLab report lists marks, strengths, weaknesses and revision topics. Marks are suggestions until a teacher accepts them.

3.5 Project features

3.5.1 Question paper and answer-key generation

Teachers upload or type a paper, and the engine splits it, assigns marks and generates keys as a background job whose progress is shown on screen. Key correctness and generation time were not measured.

3.5.2 Answer-sheet evaluation

Students choose a published paper and upload a script.

The engine extracts and evaluates it as a background job while the client polls for progress, after which the submission is finalised. Stored status values are processing, evaluated, reviewed and failed. Response time and failure frequency were not measured.

3. 5. 3 Question-wise feedback and PDF report

The result screen shows total marks, percentage, grade and pass/fail status, with strengths, growth areas and topics to revise. For each question the system stores the extracted answer, marks, grade, an off-topic flag and a review flag, and a PDF report can be downloaded. Feedback text is assembled from templates using matched and missing concepts; its usefulness to students was not assessed.

3. 5. 4 Teacher review and override

The review workspace shows the OCR text beside the AI score and lets a teacher override the overall score or any single question and add comments. Whether overrides improve consistency was not tested.

3. 5. 5 Class analytics

Analytics pages show the class mean, grade distribution, hardest questions and a Bloom-level radar for evaluated submissions. Aggregate correctness with larger classes was not tested.

3. 5. 6 Choice-based questions

A paper can define groups in which only a required number of questions count, for example two of three 14-mark questions. The 50-mark total of the pilot paper is consistent with 12 of 12, 3 of 3, 2 of 3 and
1 of 2 counted questions. How the system selects among attempted questions was not independently verified.

3. 5. 7 Accounts, dashboards and learning plans

Student and teacher dashboards, profile pages and e-mail verification are implemented. A learning-plan interface and table exist, but the inspected database held no plan records, so personalised plans were not evaluated.

3. 5. 8 Local-network deployment

The server can listen on all interfaces for access from other computers, while the browser reaches the AI engine on the client's own machine at 127.0.0.1:8000, so each evaluating computer needs its own Ollama installation. Multi-user load and network performance were not tested.

3. 6 Outputs

The following figures present the supplied TwinEval AI interface outputs and its key user-facing modules. Screens were captured from a local instance with demonstration accounts; the pilot values shown are from the stored database.

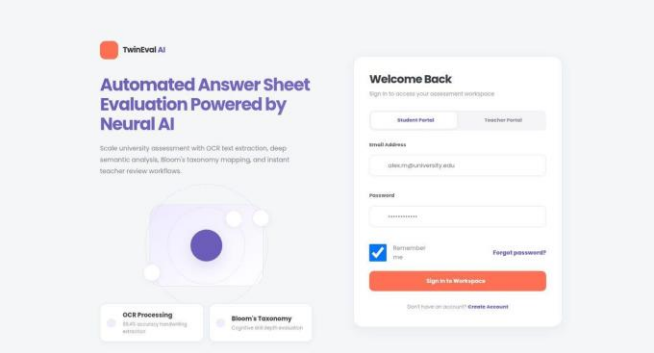


Figure 1. TwinEval AI landing and sign-in page.

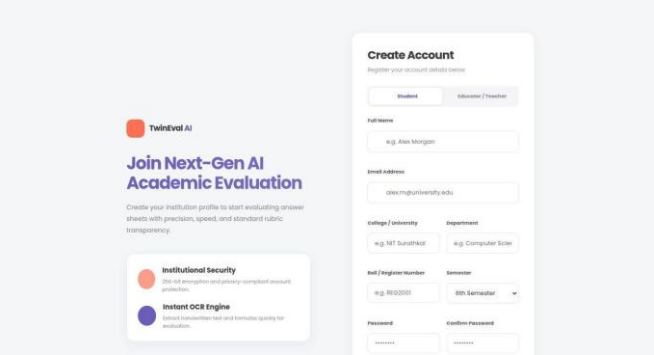


Figure 2. Account registration screen.

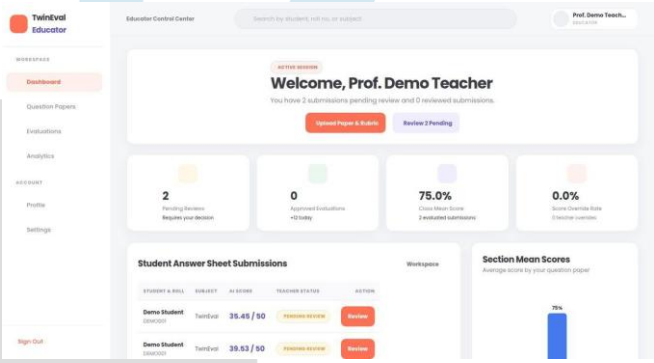


Figure 3. Teacher dashboard with pending submissions.

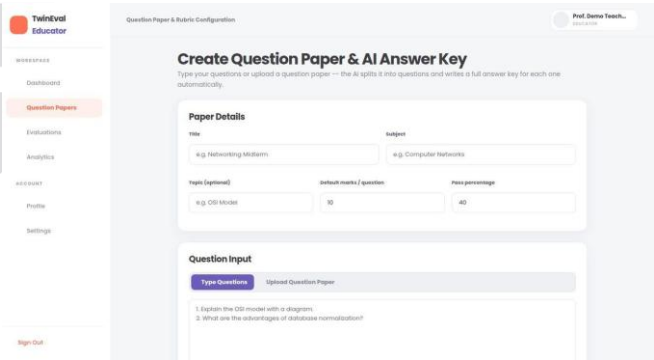


Figure 4. Question paper and answer-key setup.

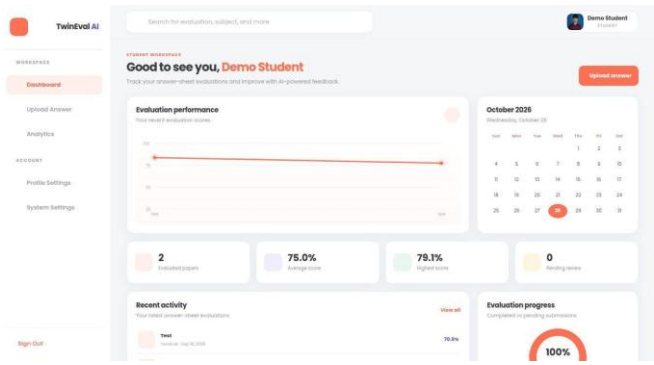


Figure 5. Student dashboard and evaluation performance.

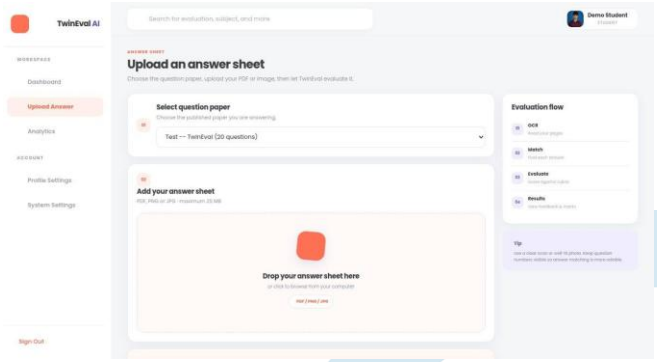


Figure 6. Answer-sheet upload screen.

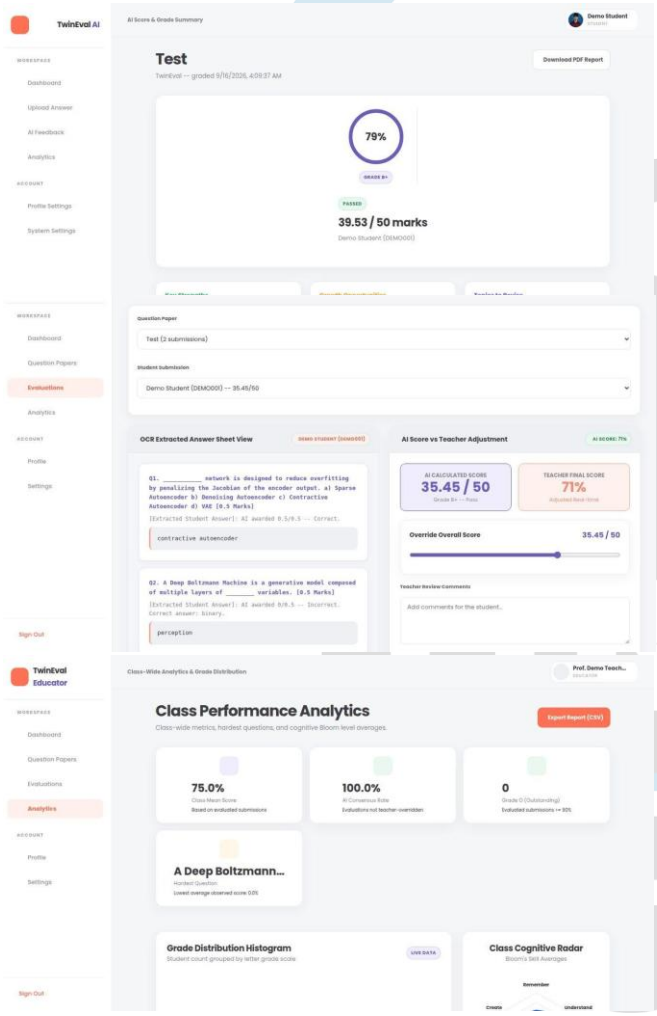


Figure 9. Class performance analytics screen.

3.7 Evaluation protocol

Two sources were used. First, a short harness written for this report and run on 29 October 2026 with AI assistance against TWINEVAL-fixed_3.zip exercised deterministic code only: model and OCR libraries were replaced by stubs, so no Gemma, embedding or OCR output was involved. It held 10 grade-boundary cases,
10 objective-answer cases, a four-question paper with mixed numbering and marks annotations, a four-answer script with mixed answer markers, and three synthetic rubric-coverage cases (full, partial, none). The cases were authored by the same assistant that inspected the code, so "expected" means the harness's own oracle, not an independent standard.

Second, the stored database was examined. It holds one published paper of 20 questions (12 of 0.5 marks, 3 of
2, 3 of 14 and 2 of 10, with 50 marks counted) and two submissions by one student account, evaluated on 16 and 18 September 2026. Both point to the same PDF file (identical file hash) and the same extracted text length of 5,051 characters, so the pair indicates run-to-run variation of the scoring stage. The source commit, the model settings of each run and any examiner marks were not recorded.

4. OUTPUTS AND TECHNICAL EVALUATION

The inspected implementation contained the components in Table 1. Table 2 reports the harness checks and the repeat-run comparison. These results describe code behaviour and run-to-run variation; they do not measure agreement with examiners.

Table 2. Component checks and repeat-run comparison.

Measure	Result	Qualification
Grade mapping	10/10	Authored boundary cases
Objective matching	9/10	One negated answer accepted
Question splitting	4/4	Questions and marks, mixed formats
Answer splitting	3/4	One unpunctuated marker missed
Rubric fallback, full / partial / none	10.0 / 5.8 / 1.2	Score out of 10 marks
Repeat-run total	39.53 vs 35.45	Same script, out of 50
Largest question spread	2.85	Q16 (14 marks): 10.75 vs 7.90

In the pilot, the two runs agreed exactly on all twelve objective items (7 correct, 3.5 of 6 marks) and on Q19 (8.75 of 10). The long-answer items differed: Q13 1.37 and 1.64, Q15 1.51 and 0.99, Q16 10.75 and
7.90, Q17 12.00 and 11.00. The totals were 39.53/50 (79.1%, B+) and 35.45/50 (70.9%, B+), a spread of 4.08
marks; both runs passed at the 40% line and fell in the same grade band.

Marks for the 14-mark items rest on a stochastic model call (temperature 0.2); two runs cannot characterise the spread. The negation miss shows that the objective path accepts "not backpropagation" for the key "backpropagation", and the unmatched marker shows that answer splitting depends on punctuation after the question number.

4. 1 Outcomes not measured

The evaluation did not measure agreement with examiner marks (for example weighted kappa [9] or mean absolute error), OCR character or word error rate, per-script processing time, language-model failure or fallback frequency, or teacher and student usability. Handwriting-style and fairness assessment were not documented.

5. OUTCOME, LIMITATIONS AND ETHICS

The checks show that the deterministic parts of the pipeline behave as coded for the authored cases. This is an engineering check, not evidence that marks agree with human examiners. The cases and their expected outcomes were written alongside the code, so generalisation to real scripts is unknown.

No examiner-scored ground truth was available: the calibration file holds no exemplars and no resolved review flags were documented. The model, OCR route and settings of the stored runs were not recorded, and the answer keys were generated by a language model and not checked by a subject teacher. One scanned script from one student, evaluated twice, cannot represent handwriting variety. OCR accuracy for different handwriting, ink, scan quality, diagrams, equations and languages other than English was not evaluated, and recognition errors pass directly into scoring.

The objective path accepts negated phrases, and relevance and confidence values are heuristics rather than calibrated probabilities. Because the review flag depends on them, a wrong score far from the pass line may not be flagged. Marks should be treated as suggestions that a teacher confirms, and the system should not support high-stakes decisions without human review.

Student scripts, names and e-mail addresses are personal data. The local-first design reduces but does not remove exposure: optional Gemini use sends content to a remote service, the database is an unencrypted file, and the inspected server code serves the uploads folder as a static path without an authentication check. No participants, institutional ethics approval or student consent were documented for the pilot records. This does not imply an ethics exemption, and the appropriate ethics statement requires author confirmation.

6. FUTURE ENHANCEMENT

Further evaluation should compare system marks with those of two or more independent examiners on a larger set of scripts from several courses, reporting weighted kappa, mean absolute error and bias by score band with the existing calibration harness. Repeated runs of the same scripts should quantify score variance, and the effect of lower temperature, a fixed seed or averaging several runs should be tested. OCR should be benchmarked separately with character and word error rates across handwriting styles, scan qualities and diagrams. The objective matcher should handle negation, and the review rules should add a variance-based flag. Access control for uploaded files, encryption of stored data and a retention policy should be added, and usability testing with teachers and students should follow appropriate ethics review and consent.

Before redistribution, document the model build, endpoint and package versions, and confirm licensing.

7. REFERENCES

1. Burrows, S., Gurevych, I., Stein, B. The eras and trends of automatic short answer grading. International Journal of Artificial Intelligence in Education 25, 60-117 (2015). https://doi.org/10.1007/s40593-014-0026-8.
2. Ramesh, D., Sanampudi, S. K. An automated essay scoring systems: a systematic literature review. Artificial Intelligence Review 55, 2495-2527 (2022). https://doi.org/10.1007/s10462-021-09977-9.
3. Reimers, N., Gurevych, I. Sentence-BERT: sentence embeddings using Siamese BERT-networks. Proceedings of EMNLP-IJCNLP 2019, 3982-3992. https://doi.org/10.18653/v1/D19-1410.
4. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M. MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers. Advances in Neural Information Processing Systems 33 (2020).
5. Zheng, L., Chiang, W.-L., Sheng, Y., et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 36 (2023). https://arxiv.org/abs/2306.05685.
6. Gemma Team. Gemma 3 technical report. arXiv:2503.19786 (2025). https://arxiv.org/abs/2503.19786.
7. Anderson, L. W., Krathwohl, D. R. (eds.). A taxonomy for learning, teaching, and assessing: a revision of Bloom's taxonomy of educational objectives. Longman, New York (2001).
8. Williamson, D. M., Xi, X., Breyer, F. J. A framework for evaluation and use of automated scoring. Educational Measurement: Issues and Practice 31(1), 2-13 (2012). https://doi.org/10.1111/j.1745-3992.2011.00223.x.
9. Cohen, J. Weighted kappa: nominal scale agreement with provision for scaled disagreement or partial credit. Psychological Bulletin 70(4), 213-220 (1968).
10. Memon, J., Sami, M., Khan, R. A., Uddin, M. Handwritten optical character recognition (OCR): a comprehensive systematic literature review. IEEE Access 8, 142642-142668 (2020). https://doi.org/10.1109/ACCESS.2020.3012542.
11. Ollama. API introduction and local model documentation. https://docs.ollama.com/api/introduction.
12. Kasneci, E., Sessler, K., Küchemann, S., et al. ChatGPT for good? On opportunities and challenges of large language models for education. Learning and Individual Differences 103, 102274 (2023). https://doi.org/10.1016/j.lindif.2023.102274.
13. Bradski, G. The OpenCV Library. Dr. Dobb's Journal of Software Tools (2000).
14. Node.js Foundation. Node.js documentation: SQLite (node:sqlite). https://nodejs.org/api/sqlite.html.
15. Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT). RFC 7519, IETF (2015). https://doi.org/10.17487/RFC7519.
16. Artifex Software. PyMuPDF documentation. https://pymupdf.readthedocs.io.
17. Shermis, M. D., Burstein, J. (eds.). Handbook of automated essay evaluation: current applications and new directions. Routledge, New York (2013).

18. Du, Y., Li, C., Guo, R., et al. PP-OCR: a practical ultra lightweight OCR system. arXiv:2009.09941 (2020). https://arxiv.org/abs/2009.09941.
19. Bloom, B. S. (ed.). Taxonomy of educational objectives: the classification of educational goals. Handbook I: Cognitive domain. David McKay, New York (1956).
20. Landis, J. R., Koch, G. G. The measurement of observer agreement for categorical data. Biometrics 33(1), 159-174 (1977). https://doi.org/10.2307/2529310.
21. Page, E. B. The imminence of grading essays by computer. Phi Delta Kappan 47(5), 238-243 (1966).
22. Vaswani, A., Shazeer, N., Parmar, N., et al. Attention is all you need. Advances in Neural Information Processing Systems 30 (2017). https://arxiv.org/abs/1706.03762.